

Agenda.

- How to approach Schema Design.
- MVC
- Design TicTacToe.

How to approach Schema Design.

1:1 : Id of 1 side on other side.

1:M : Id of 1 side on M side.

M:1 : Mapping table.

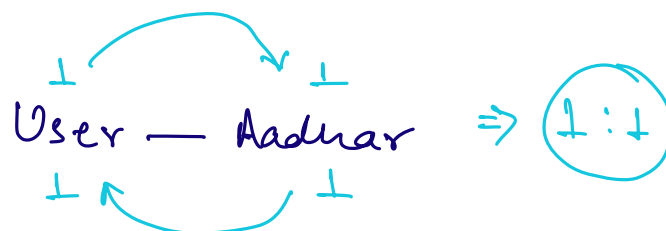
User

id	name	...	aadhar no

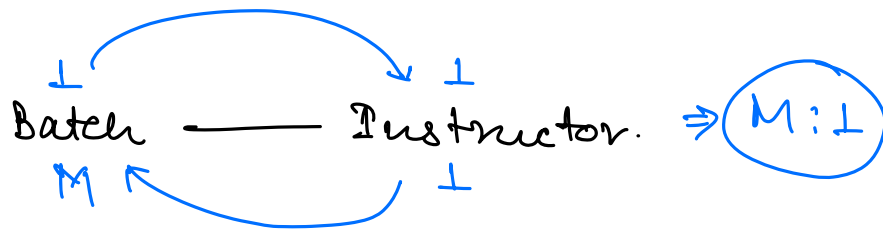
OR

Aadhar

aadhar no	...	userid



#



batches

			ins_id
id	name	...	<ins_id>

✓

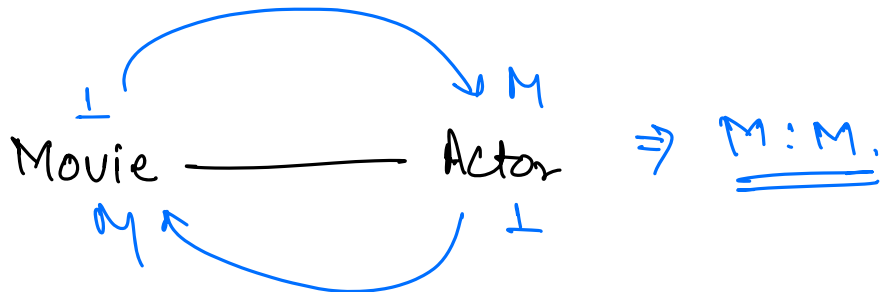
OR

instructor

		list<Batches>
100	Deepak	201, 202

$\Rightarrow$ Id of ① side on ② side.

#



movies

		list<Actor>
--	--	------------------------------

actors

		list<Movie>
--	--	------------------------------

Mapping table.

movies-actors

movie_id	actor_id

⇒ Overview
Requirement Gathering
Class Diagram
Schema Design
Code.

For every class in the class diagram, create a table in the Database.

Class Movie {
id
name
release_year
language
list < Actor >

≡



Another class.

Class Actor {
id
name
|
|
|
|

≡

movies.

id	name	release-year	lang	

⇒ For every primitive attribute, create a column in the table as it is.

⇒ for every Non primitive attr, find out the relⁿ b/w the entities & apply the corresponding rule

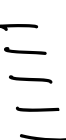
movies-actors

--	--

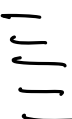
How to Code?

⇒ Project Structure.

/models/ 

/Controllers/ 

/services/ 

/factory/ 

⇒ First code all the Models.

⇒ Start implementing requirement by requirement.

MVC Architecture



Model

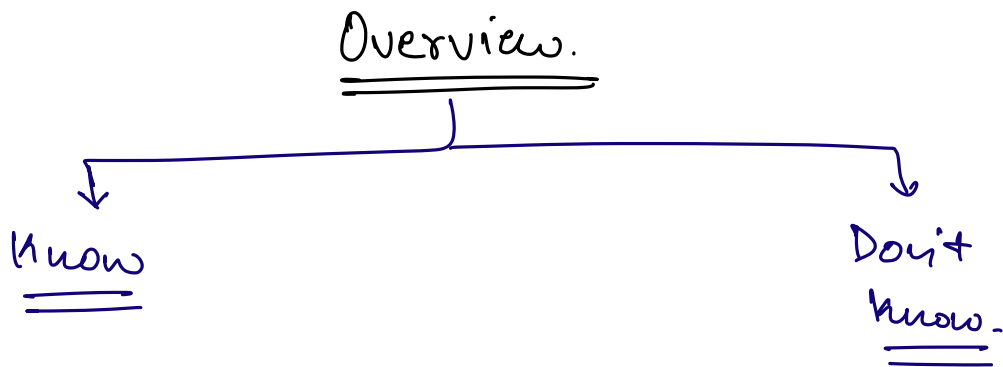
View

Controller.



Model.

Design TicTacToe.



⇒ Complete working solⁿ.

⇒ Do we need to persist data?

↳ No.

Requirements.

- ↳ Suggest features for the system.
- ↳ Clarify edge/corner cases.

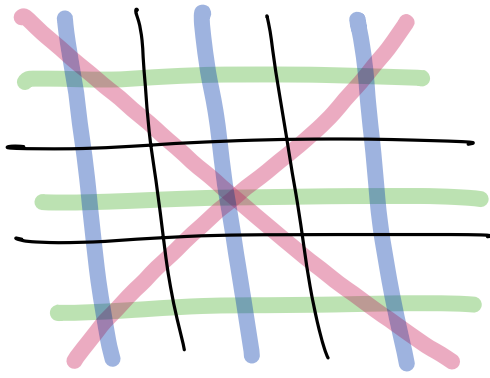
- 1) Size of the board is $N \times N$.
- 2) No. of players = $N-1$.
- 3) Each player can choose their own symbol at the start of the game but No 2 players should have the same symbol.
- 4) Will there be any BOT? Yes
- 5) Only 1 bot is allowed per game.
- 6) Bot will have different difficulty levels.
- 7) Who will make the first move?



⇒ At the start of the game, we'll randomize the list of players.

8) How to decide the winner?

⇒ A player wins the game if they get same symbol across any row | col | diagonal.



⑨ Game ends as soon as someone wins the game (or) game DRAWs.

↓
Board is full.

⑩ UNDO feature.

- Global Undo
- Anyone can press the UNDO button.
- Removes the latest move from the board.

How to implement UNDO feature?

Move.

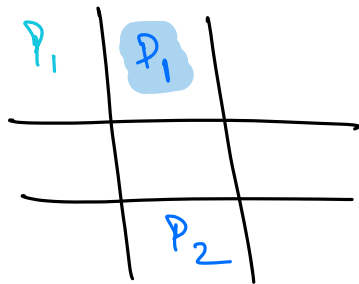
- Which player played the move & where?
- Player & location.

$$\textcircled{1} \text{ List} \langle \text{Move} \rangle = \left[\left\{ \begin{matrix} P_1 \\ 0 \\ 0 \end{matrix} \right\} \left\{ \begin{matrix} P_2 \\ 2 \\ 1 \end{matrix} \right\} \left\{ \begin{matrix} P_1 \\ 0 \\ 1 \end{matrix} \right\} \right]$$

i) Store all the moves in a list.

ii) Remove the last move from the list.

iii) Mark the cell empty [Remove the move from the board as well.].



⇒ Removing the last move from board can be difficult in games like chess.

$$\textcircled{2} \text{ List} \langle \text{Move} \rangle = \left[\begin{array}{c} \{ P_1 \\ 0 \\ 0 \} \end{array} \begin{array}{c} \{ P_2 \\ 2 \\ 1 \} \end{array} \begin{array}{c} \{ P_1 \\ 0 \\ 1 \} \end{array} \right]$$

Nth

⇒ Instead of removing Nth move from the board, we can execute first N-1 move from the beginning.

i) Reset the board.

ii) Redo the first (N-1) moves.

iii) Remove Nth move from the list of Move.

⇒ TC would be HIGH.

③ Store the state/snapshot of the board after every move.

$$\text{List} \langle \text{Board} \rangle : \left[\begin{array}{|c|c|c|} \hline & & \\ \hline \times & & \\ \hline & & \\ \hline \end{array}, \begin{array}{|c|c|c|} \hline & & 0 \\ \hline \times & & \\ \hline & & \\ \hline \end{array}, \begin{array}{|c|c|c|} \hline & & 0 \\ \hline \times & & \\ \hline & & \times \\ \hline \end{array}, \begin{array}{|c|c|c|} \hline & & 0 \\ \hline \times & 0 & \\ \hline & & \times \\ \hline \end{array} \dots \right]$$

↑
Current State.

⇒ Easy to implement.

⇒ Good TC

⇒ SC ↑.

———— * ————

TTT { Class Diagram.
Implementation

{ JAVA ⇒ OOPS
Concurrency.
Exception Handling
Streams & Lambdas
Collections.